Introduzione all'Econometria Spaziale

Laboratorio Python

Dott. Niccolò Salvini

Università Cattolica del Sacro Cuore – Roma

3 Marzo 2026

Agenda

1. Breve recap: cosa abbiamo visto in R
2. Da R a Python: pacchetti e funzioni
3. Mappatura della sintassi
4. Chi fa meglio cosa?
5. Notebook Colab disponibili

Dataset utilizzati

File	Tipo	Descrizione
`tanzania.parquet`	Areale	DHS 2022 – indicatori demografici e sanitari
`kc_house.parquet`	Puntuale	King County – prezzi delle abitazioni
`kc_grid.parquet`	Griglia H3	KC – prezzi medi su esagoni (res. 8)
`italian_provinces.parquet`	Areale	Province IT – NEET e formazione
`visium_hne_points.parquet`	Puntuale	Dati Visium HNE (spatial transcriptomics)

Tutti i dataset sono in formato **GeoParquet** e caricabili con una riga di codice sia in R che in Python.

Percorso analitico in R

Dati e visualizzazione

- Caricamento dati spaziali (shapefile → parquet)
- Mappe tematiche: coropletiche, puntuali, a griglia H3
- Aggregazione punti su griglia esagonale

Analisi spaziale

- Matrice dei pesi **W** (Queen, kNN)
- Moran's *I* globale
- LISA e Moran scatterplot
- OLS → diagnostica → SEM/SAR/SARAR
- Impatti spaziali (diretto, indiretto, totale)

Script R – riepilogo

#	**Script e contenuto**
1	`1_spatial_data.R` – caricamento e preparazione dati
2	`2_maps.R` – mappe tematiche (areali, puntuali, H3)
3	`3_lisa.R` – Moran's *I*, LISA, Moran scatterplot
3b	`3_lisa_h3.R` – LISA su griglia H3
3c	`3_lisa_visium_hne.R` – LISA con pesi kNN
4	`4_spatial_model.R` – OLS + Moran test + SEM
5	`5_spatial_model_selection.R` – LM test, SAR, SEM, SARAR, impatti

$\Downarrow$ **Stessa identica struttura in Python (7 notebook Colab)**

Mappatura dei pacchetti

Funzione	R	Python
Dati spaziali (vettori)	`sf`	`geopandas`
Lettura/scrittura parquet	`sfarrow`	`geopandas` + `pyarrow`
Manipolazione dati	`dplyr`	`pandas`
Pesi spaziali	`spdep`	`libpysal`
Autocorrelazione (Moran, LISA)	`spdep`	`esda`
Modelli spaziali	`spatialreg`	`spreg`
Grafici e mappe	`ggplot2`	`matplotlib`
Griglia esagonale H3	`h3jsr`	`h3`
Classificazione mappe	(in `ggplot2`)	`mapclassify`
Plot LISA/Moran	`spdep`	`splot`

Ecosistema PySAL

In R i pacchetti principali sono due: `spdep` + `spatialreg`.

In Python l'ecosistema equivalente è **PySAL**, composto da moduli specializzati:

`libpysal`

Strutture dati: matrice dei pesi **W** (Queen, Rook, kNN, distanza)

`esda`

Analisi esplorativa: Moran's *I* globale e locale (LISA)

`spreg`

Modelli di regressione: OLS, SEM (GM_Error), SAR (GM_Lag), SARAR (GM_Combo)

`splot`

Visualizzazione: Moran scatterplot, LISA cluster map

Caricare dati spaziali

R

```r
library(sf)
library(sfarrow)

tz <- st_read_parquet(
        "tanzania.parquet")
st_crs(tz)
head(tz)
```

Python

```python
import geopandas as gpd

tz = gpd.read_parquet(
       "tanzania.parquet")
tz.crs
tz.head()
```

- In R servono due pacchetti (`sf` + `sfarrow`); in Python basta `geopandas`.
- Il CRS si accede come attributo (`.crs`) anziché con una funzione.

Mappe coropletiche

R

```r
library(ggplot2)
ggplot(tz) +
  geom_sf(aes(fill = stunting)) +
  scale_fill_distiller(
    palette = "YlOrRd",
    direction = 1) +
  theme_minimal()
```

Python

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(
            figsize=(10, 8))
tz.plot(column="stunting",
        cmap="YlOrRd",
        legend=True, ax=ax)
ax.set_axis_off()
plt.show()
```

- ggplot2 usa grammatica a layer (+); `geopandas.plot()` wrappa `matplotlib`.
- Le colormap hanno nomi simili (`YlOrRd`) in entrambi i linguaggi.

Matrice dei pesi spaziali **W**

R

```r
library(spdep)

# Queen contiguity
nb <- poly2nb(tz,
        queen = TRUE)
w  <- nb2listw(nb,
        style = "W")

# kNN
coords <- st_coordinates(
            st_centroid(tz))
nb_k <- knearneigh(coords,
          k = 5)
w_k  <- nb2listw(
          knn2nb(nb_k))
```

Python

```python
from libpysal.weights import (
    Queen, KNN)

# Queen contiguity
w = Queen.from_dataframe(tz)
w.transform = "R"

# kNN
w_k = KNN.from_dataframe(
        tz, k=5)
w_k.transform = "R"
```

- In R: 2–3 passaggi (`poly2nb` → `nb2listw`).
- In Python: un unico metodo di classe (`Queen.from_dataframe`).

Moran's *I* globale

R

```r
moran.test(tz$stunting, w)

moran.plot(tz$stunting, w,
           labels = FALSE)
```

Python

```python
from esda import Moran
from splot.esda import (
    moran_scatterplot)

mi = Moran(tz["stunting"], w)
print(mi.I, mi.p_sim)

moran_scatterplot(mi)
```

- R: funzione procedurale → restituisce direttamente il test.
- Python: approccio OOP → oggetto `Moran` con attributi `.I`, `.p_sim`.

LISA (Local Moran)

R

```r
lisa <- localmoran(
          tz$stunting, w)
# quadrant labels
mp <- moran.plot(
        tz$stunting, w)
```

Python

```python
from esda import Moran_Local
from splot.esda import (
    lisa_cluster)

lisa = Moran_Local(
          tz["stunting"], w)
lisa_cluster(lisa, tz)
```

- `Moran_Local` restituisce attributi: `.Is` (statistiche locali), `.p_sim`, `.q` (quadranti HH/LL/HL/LH).
- `splot` genera direttamente la LISA cluster map con colori standard.

Regressione OLS e diagnostica

R

```r
ols <- lm(neet ~ lifelong,
          data = prov)

# Moran test residui
moran.test(residuals(ols), w)

# LM test
lm.LMtests(ols, w,
  test = c("LMerr", "LMlag",
           "RLMerr", "RLMlag"))
```

Python

```python
from spreg import OLS

ols = OLS(y, X,
      w=w, spat_diag=True,
      name_y="neet",
      name_x=["lifelong"])

print(ols.summary)
# include: Moran's I,
# LM_err, LM_lag,
# RLM_err, RLM_lag
```

- In R: `lm()` + funzioni separate per ogni diagnostica.
- In Python: `spreg.OLS` con `spat_diag=True` → tutto nel `.summary`.

Modelli spaziali

R

```r
library(spatialreg)

# SEM
sem <- errorsarlm(
         neet ~ lifelong,
         data = prov, listw = w)

# SAR (Spatial Lag)
sar <- lagsarlm(
         neet ~ lifelong,
         data = prov, listw = w)

# SARAR
sarar <- sacsarlm(
           neet ~ lifelong,
           data = prov, listw = w)
```

Python

```python
from spreg import (
    GM_Error, GM_Lag,
    GM_Combo_Het)

# SEM
sem = GM_Error(y, X, w=w)

# SAR (Spatial Lag)
sar = GM_Lag(y, X, w=w)

# SARAR
sarar = GM_Combo_Het(
          y, X, w=w)
```

- R usa ML (Maximum Likelihood); Python (spreg) usa GMM (Generalized Method of Moments)

Impatti spaziali (SAR)

R

```r
imp <- impacts(sar, listw = w)
summary(imp, zstats = TRUE)
# Direct, Indirect, Total
```

Funzione built-in: `impacts()` gestisce tutto.

Python

```python
import numpy as np
from scipy import sparse

rho = sar.betas[-1]
W = w.sparse
n = W.shape[0]
I = sparse.eye(n)
S = np.linalg.inv(
      (I - rho * W).toarray())

bk = sar.betas[1]
direct = np.mean(np.diag(S))*bk
total  = np.mean(S.sum(1))*bk
indirect = total - direct
```

- `spreg` non ha una funzione `impacts()` $\rightarrow$ calcolo manuale via $(I - \rho W)^{-1}$.
- In R basta una riga; in Python serve algebra matriciale esplicita.

Chi fa meglio cosa?

Ambito	**R**	**Python**
Manipolazione dati spaziali	≈	≈
Visualizzazione / mappe	✓✓	✓
Costruzione pesi W	✓	✓✓
Autocorrelazione (Moran, LISA)	✓	✓✓
Diagnostica spaziale (LM test)	✓	✓✓
Modellistica (SEM, SAR, SARAR)	✓✓	✓
Impatti spaziali	✓✓	–
Scalabilità / big data	✓	✓✓
Integrazione ML / deep learning	–	✓✓

In sintesi: R eccelle nella modellistica spaziale (ML, impatti, formula syntax). Python è più agile nella diagnostica, nella costruzione dei pesi e nell'integrazione con l'ecosistema data science.

Notebook disponibili

#	Notebook	Contenuto
1	`1_spatial_data.ipynb`	Caricamento e preparazione dati
2	`2_maps.ipynb`	Mappe: coropletiche, puntuali, H3
3	`3_lisa.ipynb`	Moran's I globale + LISA
3b	`3_lisa_h3.ipynb`	LISA su griglia H3
3c	`3_lisa_visium_hne.ipynb`	LISA con pesi kNN
4	`4_spatial_model.ipynb`	OLS + Moran residui + SEM
5	`5_spatial_model_selection.ipynb`	LM test, SAR, SEM, SARAR, impatti

Tutti i link sono sul sito del corso:
`https://vincnardelli.github.io/spec/`